

记账项目程序设计报告

小组：letsgo

组员：赵博望 2500017414 章向前 2500017457

一、功能介绍

1、基础账目管理

系统提供了完整的账单生命周期管理功能，支持用户对日常财务数据进行录入、修改、删除和展示。

(1) 账单录入：每笔账目包含金额、收支类型（收入/支出）、分类、日期以及备注说明。

(2) 收支精细分类：在新建或编辑账目时，分类下拉框会根据用户选择的“收入”或“支出”类型进行动态切换联动。例如：选择“收入”时，可选基本工资、奖金等；选择“支出”时，自动加载餐饮、购物、娱乐等分类。

(3) 直观的列表展示与操作：主界面采用 QTableView 组件结合 QStandardItemModel 进行账目数据的直观展示，支持单行选中后的修改与删除操作。

2、多条件账目检索与数据导出

系统设计了灵活的筛选与数据导出模块：

(1) 局部条件检索：

用户可组合多种条件对账单进行精细化筛选，包括：关键词匹配（模糊查询备注），分类筛选，金额区间限制（通过最大/最小双输入框限制区间）

(2) 多维度 CSV 导出：

系统支持将查询到的普通账单以标准 CSV 格式导出，文件头部自动写入 UTF-8 BOM，以防止在 Microsoft Excel 中打开时出现中文乱码。导出范围支持：导出全部账目；导出指定单月账目；导出自定义的月份起止范围。

3、自动统计分析与数据可视化

为了给用户提供直观的消费反馈，系统集成了数据可视化模块：

(1) 多维度数据汇总：系统可按“年/月”或“单日”统计总收入、总支出，并在界面上方醒目展示“本月净结余”（若结余为负，则自动切换为红色警告样式）。

(2) 双饼图并排分析：利用 Qt Charts 组件在统计页面并排渲染“收入来源占

比”与“支出分类占比”两个饼图。图表数据来源于底层的 SQL 分组查询，支持动画效果，使用户对自己的资金流向一目了然。

4、预算看板与储蓄目标管理

系统引入了科学的财务控制机制，帮助用户规避超支风险：

（1）消费限额与储蓄目标设定：用户可以通过弹窗自主配置“每月消费上限”与“每月储蓄目标”。

（2）双进度条实时渲染：消费限额进度条反映本月已支出金额占预算的比例。若支出超过上限，进度条将变为红色，并伴有超额警告文字。储蓄目标进度条反映“本月总收入 - 总支出”的实际结余相对于储蓄目标的达成进度。若顺利达标，进度条将变为绿色，提示用户目标已达成。

5、周期性账单模板与自动生成

针对房租、月薪、订阅服务等具有固定时间周期发生的收支，系统设计了自动生成周期性账单的功能：

（1）账单模板创建：在添加账目时，用户可勾选“周期账单”，并通过二级弹窗选择循环频率（日度账单、月度账单、年度账单）。

（2）智能补记机制：系统启动或用户手动刷新时，后台会自动扫描周期账单模板。若当前日期已超过“下一次生成日期”，系统会自动创建一条普通账目。为了防止用户长时间未打开软件导致漏记，系统支持使用循环机制一次性补齐中间漏掉的多条账单。

6、智能 AI 记账与财务推演

本系统的一大特色是支持外接大语言模型，实现了多模态 AI 功能：

（1）多模态 AI 对话与识单记账：通过 `recognizeReceiptWithAi` 接口，系统能够自适应两种交互模式：

①纯文本对话记账：用户输入一段文字（如“昨天吃麦当劳花了 35 元”），AI 会根据当前日期基准智能推导真实日期、金额、商户和分类。

②图片拖拽识单记账：用户可将账单、收据或发票图片直接拖拽到对话框中，后台会自动将图片转为 **Base64** 格式发送给多模态大模型，智能提取出账单中的各项数据。

同时为防止大模型自定义分类导致账目混乱，系统在提示词中加入了当前软件支持的合法分类列表，强制 AI 输出的结果与系统内预置的分类对齐。

系统还提供了 API 密钥配置弹窗。为了保护用户的个人隐私与密钥安全，系统采用了基于固定盐值与机器唯一 ID 混合生成的 SHA-256 密钥,对 API Key 进行 XOR 加密后配合 Base64 存储至 QSettings 注册表/配置文件中。

(2) AI 财务预测报告:

- ①消费趋势预测：采用指数权重移动平均算法处理历史消费序列。在预测前，系统会自动使用稳健统计学的 Median-MAD 算法 ($\text{Median} + 3 * 1.4826 * \text{MAD}$) 剔除偶发性大额消费，确保预测模型不受极端噪点干扰。最终结合“历史弹性消费均值”与“下月固定周期账单”推演出本月剩余时间的预估消费及最终落点。
- ②储蓄达成日期预测：通过分析历史日常收益率与支出率，系统推演本月剩余天数的日均净增量，并通过逐日递增模拟算法，精确推演并预测出用户在哪一天能够正式达成设定的储蓄目标（如“预计将在本月 15 日前后突破目标”）。如果储蓄趋势为负，则系统会发出亏损警告。

二、项目各模块与类实现细节

本系统在架构上严格遵循了“数据层与表现层分离”的原则。前端 UI 负责事件响应与数据渲染，后端统一管理数据库状态与网络通信，两者通过 Qt 的信号与槽机制进行完全解耦的异步通信。以下为各核心模块的类设计细节：

(一) 全局数据结构定义 (global.h)

- 为了保证各模块间数据传递的规范性，系统抽离了底层的核心数据结构。
- 1、AccountRecord：这是系统最核心的数据载体。不仅包含了普通账目所需的基础字段 (id, amount, type, category, date, remark)，还包含了周期账单的模板属性 (isPeriodic, periodType, nextDate)。这使得普通账目与周期模板可以共用同一张数据库表，写代码时，不管是添加、删除、修改还是查询，两类账单都能用同一套方法来处理，不用为它们分别写两套复杂的代码。。
 - 2、ExpensePredictionResult / SavingsPredictionResult：这两个结构体封装了 AI 财务预测的运算结果。结构体中保存了历史推演的输入参数（如 historyMonths, alpha , outlierThreshold ）以及最终推演结论（如 predictedRemaining, dailyNetSavingsRate），方便前端直接读取缓存渲染报告。
- (二) 核心数据与业务逻辑引擎 (DataManager 类)

DataManager 是系统的核心数据管理与业务逻辑处理中心，承担了所有的

持久化存储、复杂数学计算以及网络通信任务。

1、单例模式 (Singleton Pattern): 通过 `static DataManager* instance()` 构造唯一的实例。确保整个程序运行期间只有一个 SQLite 数据库连接, 有效避免了多线程或多窗口同时读写数据库引发的锁冲突。

2、响应式数据更新机制: 每当执行 `addRecord()`、`deleteRecord()`、`updateRecord()` 等修改数据库的操作成功后, `DataManager` 都会自动发射自定义信号 `emit dataChanged()`。前端的所有看板和表格只需监听此信号, 即可实现界面的自动化重绘。

3、本地数据加密存储设计: 在保存用户配置的 API Key 时, 没有采用明文存储, 而是通过 `getEncryptionKey()` 方法, 利用了 `QSysInfo::machineUniqueId()` (机器物理唯一标识) 配合自定义盐值生成 SHA-256 散列, 再使用 XOR 结合 Base64 算法对配置信息进行加密 (`encryptString / decryptString`), 极大提升了本地数据的安全性。

```
// 生成加密密钥
QByteArray DataManager::getEncryptionKey() const
{
    QString salt = "YourFixedSalt_ChangeMe!@#";
    QString machineId = QSysInfo::machineUniqueId();
    QString combined = salt + machineId;
    return QCryptographicHash::hash(combined.toUtf8(), QCryptographicHash::Sha256);
}

// XOR + Base64 加密
QString DataManager::encryptString(const QString& plain) const
{
    QByteArray key = getEncryptionKey();
    QByteArray data = plain.toUtf8();
    for (int i = 0; i < data.size(); ++i) {
        data[i] = data[i] ^ key[i % key.size()];
    }
    return data.toBase64();
}
```

4、稳健统计学与 EWMA 算法实现: 在 `calculateOutlierThreshold()` 中, 代码手写了基于绝对中位差的异常值剔除算法 ($\text{median} + 3.0 * 1.4826 * \text{mad}$)。清理后的数据传入 `calculateEWMA()`, 通过指数权重移动平均公式 $\text{forecast} = \alpha * \text{values}[i] + (1.0 - \alpha) * \text{forecast}$ 赋予近期消费更高权重, 从而实现科学预测。

```

    return median + 3.0 * 1.4826 * mad;
}

// 对清洗后的月度弹性消费做指数权重移动平均
double DataManager::calculateEWMA(const QList<double>& values, double alpha) const
{
    if (values.isEmpty()) {
        return 0.0;
    }

    alpha = std::max(0.01, std::min(alpha, 1.0));

    double forecast = values.first();
    for (int i = 1; i < values.size(); ++i) {
        forecast = alpha * values[i] + (1.0 - alpha) * forecast;
    }

    return forecast;
}

```

（三）主控视图控制器（Widget 类）

Widget 是系统的主窗口，利用 QStackedWidget 管理多个页面（明细、图表、查询）。

1、模型-视图渲染 (QStandardItemModel+QTableView)：在 updateTable() 与 displaySearchResults() 方法中，放弃了简易的 QTableWidget，转而使用更为标准的 QStandardItemModel 组装数据，再绑定给 QTableView，实现了严格的数据与 UI 解耦。

2、图表高内聚封装 (Qt Charts)：在 initChartView() 及 updateStatisticsPage() 中，通过 QPieSeries 和 QChart 动态构建图表实例，利用 QChartView 的 setObjectName 精准寻址并替换旧图表。同时开启了 QChart::SeriesAnimations，为用户提供丝滑的图表加载动画。

3、预算看板动态更新：updateBudgetUI() 方法中大量使用了动态 setStyleSheet。通过计算支出与限额的比例，动态切换进度条颜色（超支变红 #FF4D4F，达标变绿 #52C41A），实现了用户交互反馈。

（四）弹窗交互模块群 (Dialog Classes)

系统将各个具体的操作封装成了独立的高内聚弹窗类（继承自 QDialog），并通过模态框 (Modal) 的方式返回 QDialog::Accepted 等状态，与主窗口交互。

1、账单编辑模块 (RecordDialog)：

①信号槽联动：在 initTypeAndCategorySelectors() 中，通过监听 comboBox_type 的 currentIndexChanged 信号，动态 clear() 并重新加载子分类

下拉框，实现了丝滑的级联菜单。

②一键 AI 填充：提供了“AI 智能识别”按钮，内部实例化 AiDialog。当 AI 识别完毕后，将解析得到的 AccountRecord 直接反填进 UI 控件（金额、分类、日期等），大幅提升录入效率。

2、外接大模型交互模块 (AiDialog):

①图片拖拽功能实现：外部文件拖拽导入功能主要依赖于对 dragEnterEvent、dragMoveEvent 和 dropEvent 三个事件处理函数的具体编写，以及与类成员变量的协同工作。首先，当前窗口类在构造函数中通过调用 setAcceptDrops(true) 开启接收外部文件的权限；随后，当外部文件滑入窗口并在其上方移动时 dragEnterEvent 与 dragMoveEvent 的实现逻辑会通过检测内置 QMimeData 对象的 hasUrls() 属性，判断拖放载荷中是否包含文件路径，并通过持续调用 acceptProposedAction() 维持指针的“可放置”接收状态；最后，在用户释放鼠标并触发的 dropEvent 中，函数内部解析获取到的 URL 列表，通过 toLocalFile() 接口提取出本地绝对路径，并最终调用 QImage::load(localPath) 将解码后的图像像素数据直接读取并填充到类的私有成员变量 m_currentImage 中，完成单据图像数据在内存层面的固化载入。

```
void AiDialog::dropEvent(QDropEvent *event)
{
    QList<QUrl> urls = event->mimeTypeData()->urls();
    if (!urls.isEmpty()) {
        QUrl url = urls.first();
        QString localPath = url.toLocalFile();

        if (!localPath.isEmpty()) {
            if (m_currentImage.load(localPath)) {
                QFileInfo fileInfo(localPath);
                ui->textEdit_prompt->append(QString("\n[📎] 已成功关联图片: %1").arg(fileInfo.fileName()));

                ui->label_status->setText("✅ 单据图片载入成功！可点击下方按钮开始识别。");
                ui->label_status->setStyleSheet("color: #27ae60; font-weight: bold;");
            } else {
                ui->label_status->setText("❌ 图片文件损坏或格式不支持！");
                ui->label_status->setStyleSheet("color: #e74c3c;");
            }
        }
    }
}
```

②异步网络请求与 JSON 解析：组装包含提示词和 Base64 图片数据的复杂 JSON，利用 QNetworkAccessManager 发送异步 HTTP POST 请求。在回调槽中，使用 QJsonDocument 剥离大模型返回的冗余 Markdown 标记（如 ``json`），精准解析出账目结构体返回给上层。

```

QString finalPrompt;
if (hasImage) {
    finalPrompt = "你是一个精密的记账单据/发票/收据 OCR 智能识别引擎。请帮我分析这张账单图片。\\n";
} else {
    finalPrompt = "你是一个精密的纯文本记账智能解析引擎。请帮我分析用户输入的记账文本陈述。\\n";
}

finalPrompt += QString(
    "\\n【▲ 核心时间基准 ▲】\\n"
    "当前的现实世界真实日期今天是：%1\\n"
    "注意：如果用户在文字或单据中提到了“昨天”、“前天”、“上周五”、“大前天”等相对时间词汇，"
    "你必须以这个【当前现实世界真实日期】为绝对基准，进行日期的加减推导计算！\\n"
    "（例如：今天是 2026-06-05，用户说“昨天”，则 JSON 中的 date 必须精准计算并填入 \\\"2026-06-04\\\"）。\\n\\n"
).arg(todayStr);

finalPrompt += QString(
    "你必须严格返回一个合法的纯 JSON 格式字符串，绝对不能包含任何 markdown 的 \\\"\\\"json 包裹标记，也不要包含任何多余的解释文字。\\n\\n"
    "JSON 的格式规范必须严格如下：\\n"
    "{\\n"
    "  \\\"amount\\\": 123.45, // 数值类型，识别到的实际消费/收入总金额\\n"
    "  \\\"type\\\": \\\"EXPENSE\\\", // 字符串类型，只能是 \\\"EXPENSE\\\"（支出）或 \\\"INCOME\\\"（收入）\\n"
    "  \\\"category\\\": \\\"分类名\\\", // 字符串类型，必须从允许的分类列表中选择\\n"
    "  \\\"date\\\": \\\"%1\\\", // 字符串类型，格式统一为 yyyy-MM-dd\\n"
    "  \\\"remark\\\": \\\"商户名或商品说明\\\"\\n"
    "}\\n"
).arg(todayStr);

if (hasText) {
    finalPrompt += "\\n用户的输入内容/额外补充说明：\" + userPrompt;
}

if (!allowedCategories.isEmpty()) {
    finalPrompt += "\\n\\n【! 核心分类约束 !】\\n"
    "请注意：你的 JSON 中 'category' 字段的值必须且只能从以下合法列表中选择，绝不允许自定义：\\n"
    + allowedCategories.join(", ")
    + "\\n\\n如果内容无法明确对应，请一律分类为 '其他'。\";
}

```

- 3、预测报告呈现模块 (AiPredictionDialog)：负责通过 loadAIReport() 调用底层的 updateAIExpensePrediction() 与 updateAISavingsPrediction()。采用 HTML 标签进行富文本排版，将复杂的推演数据以直观、高颜值的体检报告形式呈现给用户。
- 4、配置与预算模块 (ApiConfigDialog / BudgetDialog)：提供简单明了的配置入口。BudgetDialog 提供了 setInitialValues 与 getLimit/getSavings 接口，使得数据的传入和传出变得干净清晰。

三、项目总结与反思

本项目设计并实现的是一个智能个人财务管理系统。在基础架构的建设上，我们利用面向对象的设计思想，构建了清晰的模型-视图分层结构。通过单例模式设计 DataManager 类，实现了多窗口间数据库操作的高效协同与数据一致性。同时，我们实现了基于日期扫描的周期账单自动补记机制，并通过 Qt Charts 实现了动态的双饼图并排渲染，直观反馈用户的财务状况。更重要的是，我们探索了将大语言模型集成进我们的系统以实现基于文本陈述与多模态单据识别的智能记账功能，并利用“Median-MAD 稳健去噪 + EWMA 指数平滑”数学模型提供了消费与储蓄趋势的推演能力。

同时，通过本项目从头到尾的完整开发，我们在技术与工程实践上收获了许多宝贵的经验。首先在异步编程与非阻塞设计方面，我们掌握了 Qt 下基于 QNetworkAccessManager 的异步 HTTP 通信，并理解了如何利用 Lambda 表达式与信号槽机制来应对网络延迟，从而有效避免网络请求导致的主 UI 线程阻

塞，切实提升了界面的流畅度与用户体验；其次在事件流控制上，我们深入学习了 Qt 的事件分发机制，通过在窗口类中具体实现特定的事件处理逻辑（如拖放文件导入），成功打通了操作系统与桌面应用之间的数据传输通道，加深了对 GUI 系统底层交互原理的理解；最后我们在实践中初步掌握了软件工程的基本数据保护规范，在对接第三方 API 时通过硬件指纹绑定与 XOR 算法实现了本地敏感密钥的混淆加密存储。

当然，我们的项目依旧比较青涩。在功能设计上，我们并没有想出足够有创意的想法，或者说自己的能力也限制了这一点。在 UI 界面的实现上，我们经历过几次优化，但是对比市面上的一些记账软件，肯定仍存在着差距。在与用户的交互上，诸如页面的布置、提示词的出现时机等仍然有提升的空间。

不过最重要的是，我们学会了如何合作开发一个程序，如何实现项目同步的推进。前后一个月的时间里，我们经历了将一个项目不断打磨调整、将设想的功能编写实现、将先前不知道原理的功能解明，这种经历带来的经验是最难得的。

四、成员分工

章向前：后端功能的编写

赵博望：前端 UI 的设计实现

功能设计、改进由二人共同完成